P.PORTO

Angular – Conceitos Práticos e Exemplos

Programação em Ambiente Web

Índice

Angular Components

Angular Form Validation

Angular Material – Exemplos

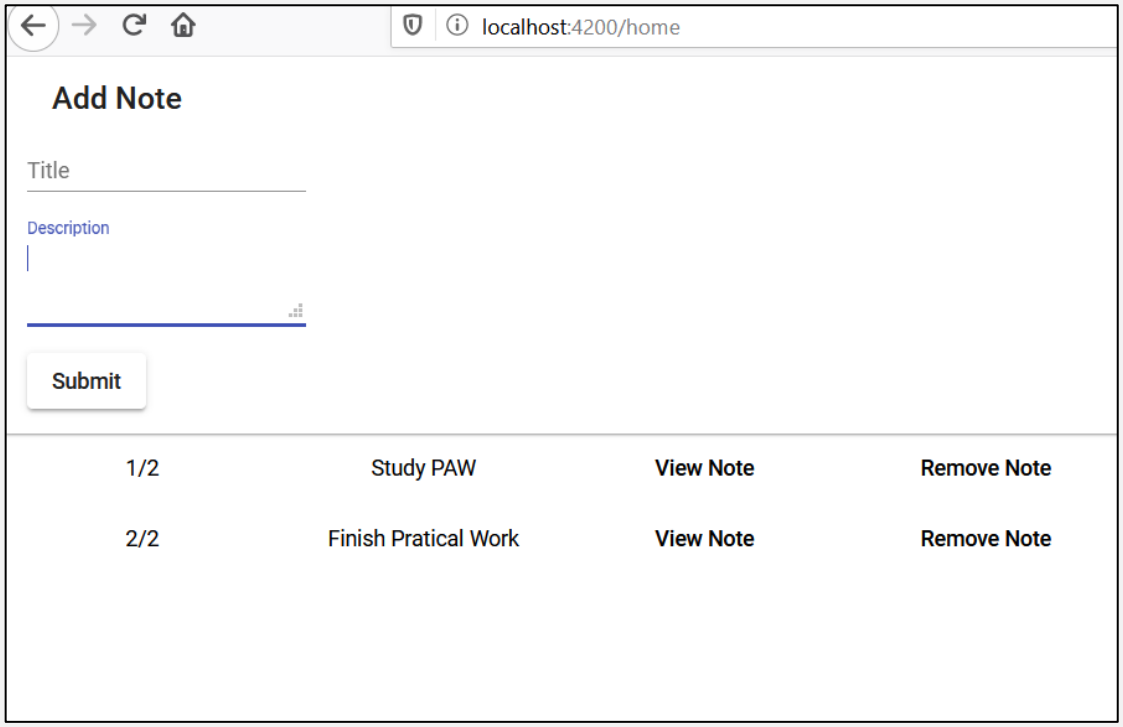
Deploy Angular no GitlabPages

Referências

Angular Components

Vamos considerar uma aplicação sobre notas

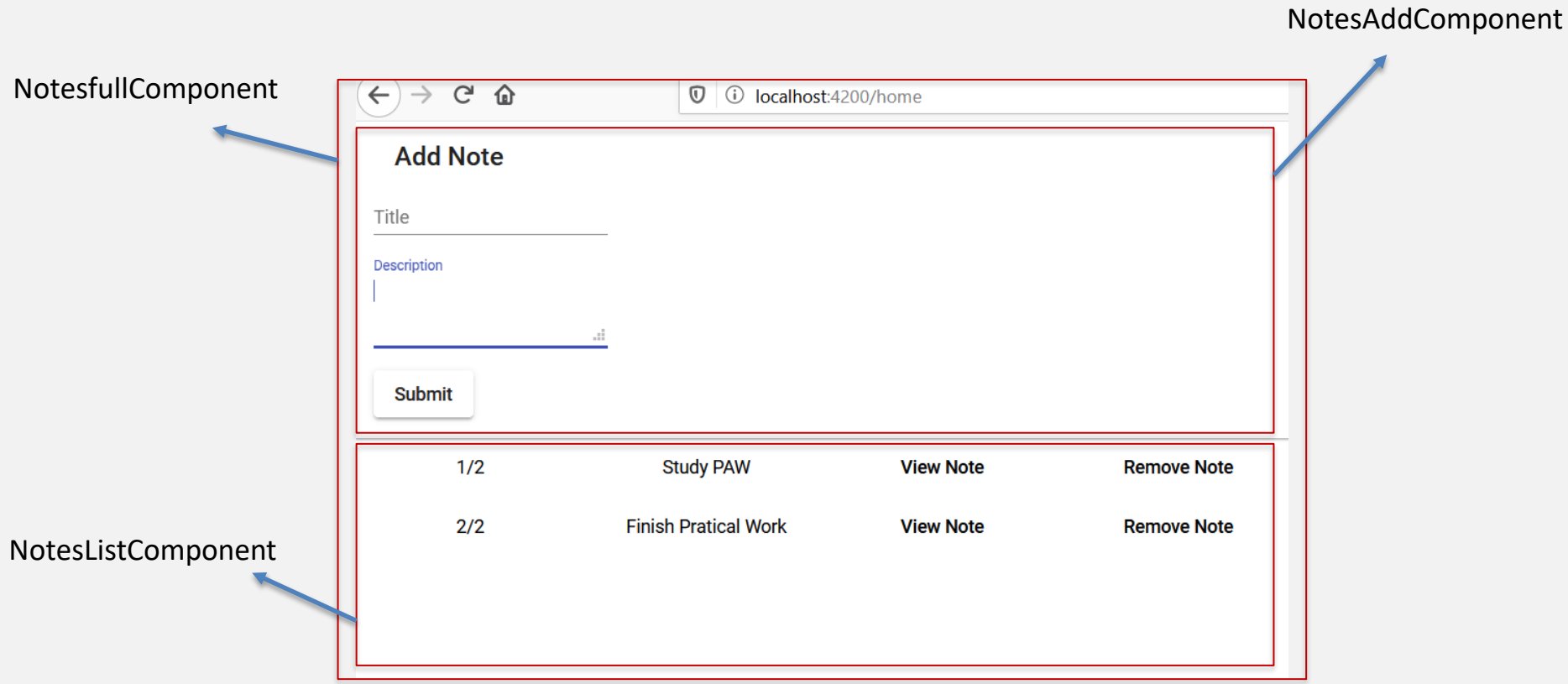
Como criar esta página?



Angular Components

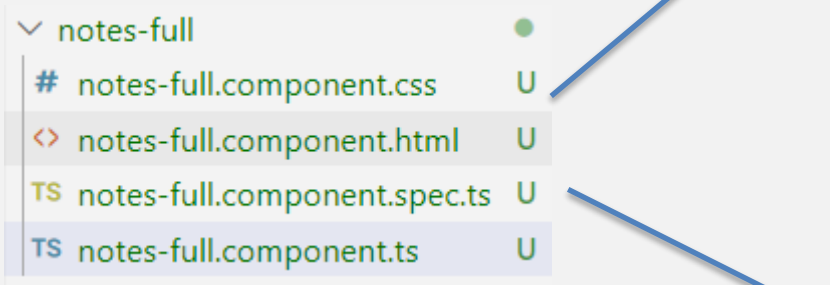
Vamos considerar uma aplicação sobre notas

Como criar esta página?



Angular Components

NotesFullComponent




```
<app-note-add></app-note-add>
<app-note-list></app-note-list>
```

```
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-notes-full',
  templateUrl: './notes-full.component.html',
  styleUrls: ['./notes-full.component.css']
})
export class NotesFullComponent implements OnInit {

  constructor() { }

  ngOnInit(): void {
  }

}
```

Angular Components

Mas como mantemos o conjunto de notas consistente entre os vários componentes?

Angular Components

```
export class Note {

    public title:string;
    public description:string;

}
```

Guardamos o estado das nossas notas num serviço em Angular

```
import { Injectable } from '@angular/core';
import { Note } from '../models/note';
import { Observable, BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class NotesServiceService {


  private _notes: Note[];
  notes : BehaviorSubject<Note[]>;

  constructor() {
    this.notes = new BehaviorSubject<Note[]>([]);
    this._notes = [];
  }

  addNote(a: Note){
    this._notes.push(a);
    this.notes.next(this._notes);
  }

  deleteNote(index: number){
    this._notes.splice(index,1);
    this.notes.next(this._notes);
  }


}
```

Angular Components

Utilizaremos uma variáveis do tipo BehaviourSubject para propagar mudanças na nossa lista de notas automaticamente pelos vários componentes que usem o serviço

BeheviourSubject uso o conceito de publish/subscribe e permite publicar alterações à nossa variável que são recolhidas pelo método subscribe nos clientes

Angular Components

NoteListComponent


```
<mat-grid-list cols="4" rowHeight="4:1">
    <ng-container *ngFor="let note of notes; index as i">
        <mat-grid-tile>{{i+1}}/{{notes.length}}</mat-grid-tile>
        <mat-grid-tile>{{note.title}}</mat-grid-tile>
        <mat-grid-tile> <button mat-button (click)="openDialog(i)">View Note</button> </mat-grid-tile>
        <mat-grid-tile> <button mat-button (click)="removeNote(i)">Remove Note</button> </mat-grid-tile>
    </ng-container>
</mat-grid-list>
```

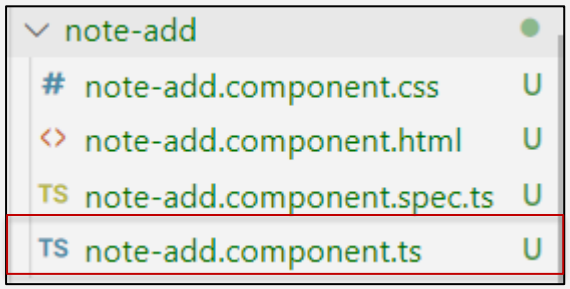
Angular Components

NoteListComponent


```typescript
@Component({
  selector: 'app-note-list',
  templateUrl: './note-list.component.html',
  styleUrls: ['./note-list.component.css']
})
export class NoteListComponent {

  notes : Note[];

  constructor(private notesService: NotesServiceService, public dialog: MatDialog) {
  }

  ngOnInit(): void {
    this.notesService.notes.subscribe((notes) =>{ this.notes = notes;
    console.log(JSON.stringify(notes))}, (err) => { console.log(err); });
  }

  openDialog(index:number): void{
    const dialogConfig = new MatDialogConfig();
    dialogConfig.disableClose = true;
    dialogConfig.autoFocus = true;
    dialogConfig.data = {
        note:this.notes[index]
    };
    const dialogRef = this.dialog.open(NotesDialogComponent,dialogConfig);
    dialogRef.afterClosed().subscribe(result => {
      console.log(`Dialog result: ${result}`);
    });
  }

  removeNote(index:number): void{
    this.notesService.deleteNote(index);
  }

}
```

Angular Components

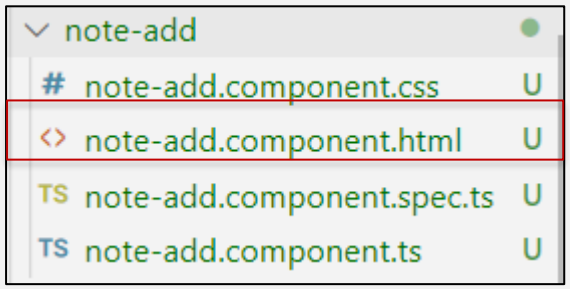
NoteAddComponent




```
@Component({
  selector: 'app-note-add',
  templateUrl: './note-add.component.html',
  styleUrls: ['./note-add.component.css']
})
export class NoteAddComponent implements OnInit {

  note : Note;

  constructor(private service: NotesServiceService) {
    this.note=new Note();
  }

  ngOnInit(): void {
  }

  onSubmit(e:Event){
    e.preventDefault();
    this.service.addNote(this.note);
    this.note = new Note();
  }

}
```

Angular Components

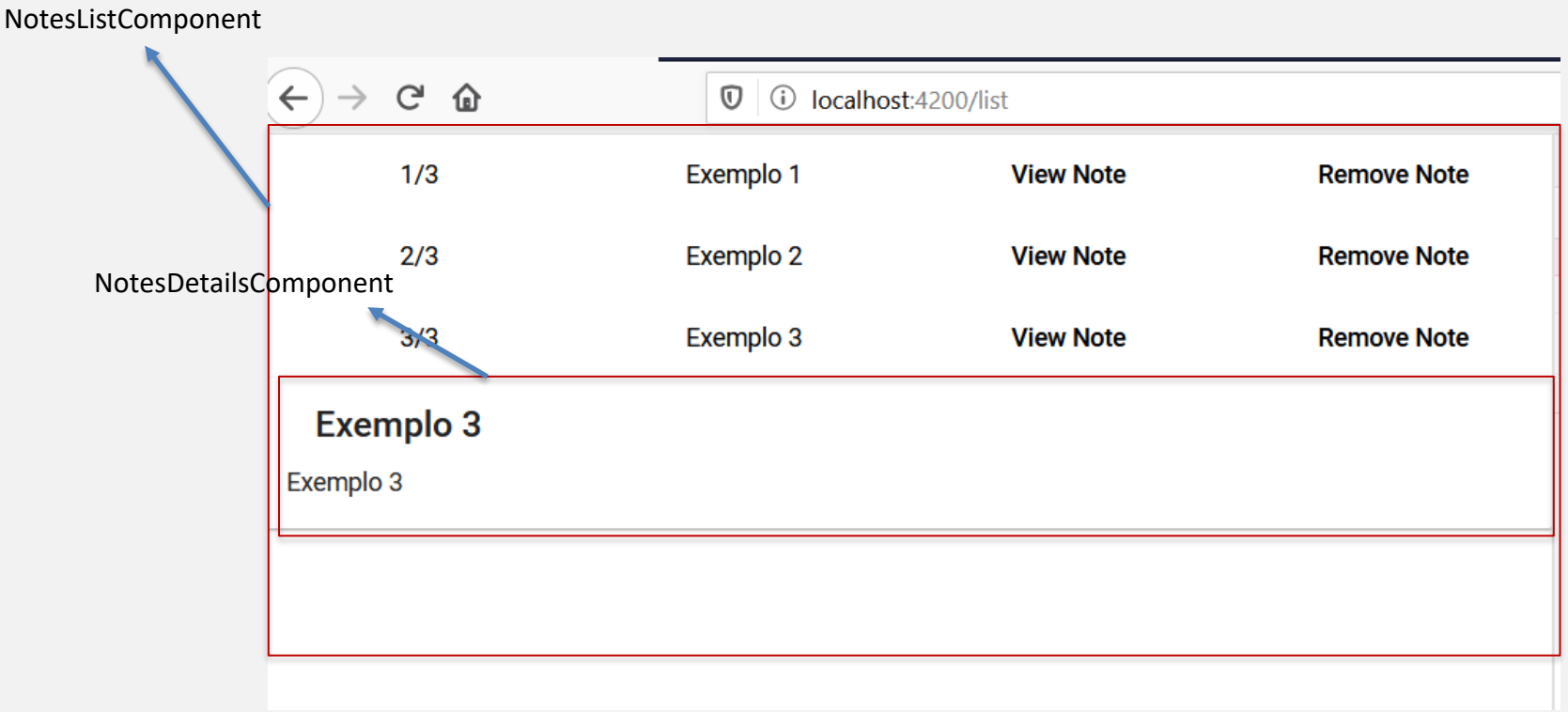
NoteAddComponent




```
<mat-card>
    <mat-card-header>
        <mat-card-title>Add Note</mat-card-title>
    </mat-card-header>
    <mat-card-content>
        <form (submit)="onSubmit($event)">
            <div class="form-group">
                <mat-form-field class="example-full-width">
                    <mat-label>Title</mat-label>
                    <input matInput placeholder="Title"class="form-control" type="text" id="title"
                    [(ngModel)]="note.title" name="title"  #title="ngModel">
                </mat-form-field>
                <!-- <label for="title">Title:</label>
                <input class="form-control" type="text" id="title"
                [(ngModel)]="note.title" name="title"  #title="ngModel">  -->
            </div>
            <div class="form-group">
                <mat-form-field class="example-full-width">
                    <mat-label>Description</mat-label>
                    <textarea matInput class="form-control" type="text" id="descritpion"
                    [(ngModel)]="note.description" name="description"  #descritpion="ngModel"></textarea>
                </mat-form-field>
                <!-- <label for="descriprion" > Description </label>
                <input  class="form-control" type="text"id="descritpion"
                [(ngModel)]="note.description" name="description"  #descritpion="ngModel">  -->
            </div>
            <button mat-raised-button type="submit" class="btn btn-success" >Submit</button>
        </form>
    </mat-card-content>
</mat-card>
```

Angular Components

E se quisermos ter uma página em que os detalhes aparecem quando selecionado na página?



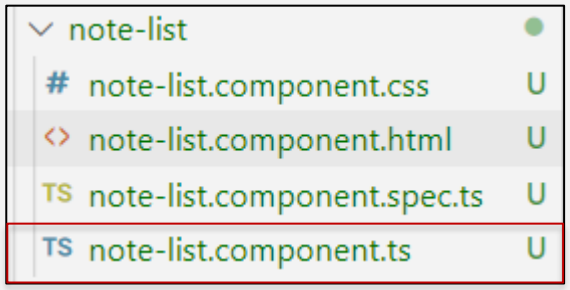
Angular Components

NoteListComponent


```
<mat-grid-list cols="4" rowHeight="4:1">
    <ng-container *ngFor="let note of notes; index as i">
        <mat-grid-tile>{{i+1}}/{{notes.length}}</mat-grid-tile>
        <mat-grid-tile>{{note.title}}</mat-grid-tile>
        <mat-grid-tile> <button mat-button (click)="selectNote(note)">View Note</button> </mat-grid-tile>
        <mat-grid-tile> <button mat-button (click)="removeNote(i)">Remove Note</button> </mat-grid-tile>
    </ng-container>
</mat-grid-list>
<app-notes-details [note]="selectedNote"></app-notes-details>
```

Angular Components

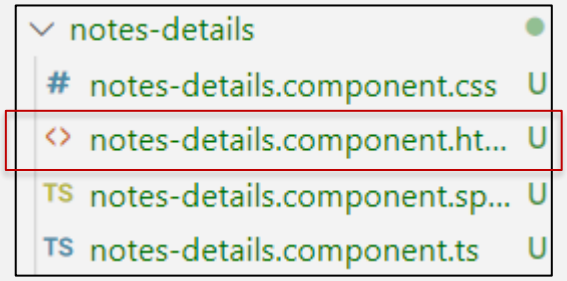
NoteListComponent




```typescript
import { Component, OnInit } from '@angular/core';
import { Note } from '../models/note';
import { NotesServiceService } from '../services/notes-service.service';

@Component({
  selector: 'app-notes-list2',
  templateUrl: './notes-list2.component.html',
  styleUrls: ['./notes-list2.component.css']
})
export class NotesList2Component implements OnInit {

  notes : Note[];
  selectedNote:Note;

  constructor(private notesService: NotesServiceService) { }

  ngOnInit(): void {
    this.notesService.notes.subscribe((notes) =>{ this.notes = notes;
    this.selectedNote=null;
    console.log(JSON.stringify(notes))}, (err) => { console.log(err); });
  }

  selectNote(note:Note){
    this.selectedNote=note;
  }

  removeNote(index:number): void{
    this.notesService.deleteNote(index);
  }

}
```

Angular Components

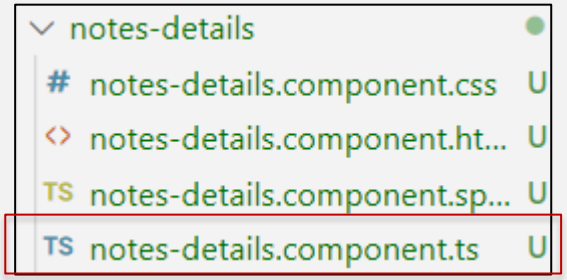
NoteDetailsComponent




```
<mat-card *ngIf="note">
    <mat-card-header>
      <mat-card-title>{{note.title}}</mat-card-title>
    </mat-card-header>
    <mat-card-content>
      {{note.description}}
    </mat-card-content>
</mat-card>
```

Angular Components

NoteListComponent




```typescript
@Component({
  selector: 'app-notes-details',
  templateUrl: './notes-details.component.html',
  styleUrls: ['./notes-details.component.css']
})
export class NotesDetailsComponent implements OnInit {

  @Input()
  note:Note;

  constructor() { }

  ngOnInit(): void {
  }

}
```

Angular Components

Quais as diferenças entre as páginas apresentadas?

Que componentes podem ser reutilizados?

Para que serve o serviço?

Angular Components

Na verdade o nosso serviço tem um pequeno problema, ele não persiste a nossa lista de notas!

De cada vez que a página é refrescada a lista de notas perde-se

Podemos usar localStorage para resolver o problema

Angular Components

Actualização do nosso serviço em Angular

```
@Injectable({
  providedIn: 'root'
})
export class NotesServiceService {


  private _notes: Note[];
  notes : BehaviorSubject<Note[]>;

  constructor() {
    this._notes = JSON.parse(localStorage.getItem('notes'));
    this._notes = this._notes==null?[]:this._notes;
    this.notes = new BehaviorSubject<Note[]>(this._notes);
  }

  addNote(a: Note){
    this._notes.push(a);
    localStorage.setItem('notes',JSON.stringify(this._notes));
    this.notes.next(this._notes);
  }

  deleteNote(index: number){
    this._notes.splice(index,1);
    this.notes.next(this._notes);
  }


}
```

Angular Form Validation

Em Angular exsitem pelo menos 2 tipos de formulários:

- ReactiveForms
- TemplateForms

Nós temos essencialmente trabalhado com TemplateForms

Angular Form Validation

	REACTIVE	TEMPLATE-DRIVEN
Setup (form model)	More explicit, created in component class	Less explicit, created by directives
Data model	Structured	Unstructured
Predictability	Synchronous	Asynchronous
Form validation	Functions	Directives
Mutability	Immutable	Mutable
Scalability	Low-level API access	Abstraction on top of APIs

Angular Form Validation

Para a utilização destes formulários é necessário declarar e importar o FormsModule

```
import { NgModule }      from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { FormsModule }   from '@angular/forms';

import { AppComponent }  from './app.component';
import { HeroFormComponent } from './hero-form/hero-form.component';

@NgModule({
  imports: [
    BrowserModule,
    FormsModule
  ],
  declarations: [
    AppComponent,
    HeroFormComponent
  ],
  providers: [],
  bootstrap: [ AppComponent ]
})
export class AppModule { }
```

Angular Form Validation

No exemplo anterior data-binding era conseguido com indicações diretamente sobre os elementos de input

```
<input matInput placeholder="Title"class="form-control" type="text" id="title"
[(ngModel)]="note.title" name="title"  #title="ngModel">
```

```
<textarea matInput class="form-control" type="text" id="descritpion"
[(ngModel)]="note.description" name="description"  #descritpion="ngModel"></textarea>
```

Angular Form Validation

Podemos adicionar validação sobre os dados de um formulário

```
<label for="firstName">First name: </label>
<input required minlength="3" maxlength="10" type="text" id="firstName" ngModel name="firstName"
    #firstName="ngModel" (change)="log(firstName)">

<div *ngIf="firstName.touched && !firstName.valid">
    <div *ngIf="firstName.errors.required">The field is required</div>
    <div *ngIf="firstName.errors.minlength">The field must have at least
        {{ firstName.errors.minlength.requiredLength }} characters</div>
    <div *ngIf="firstName.errors.maxlength">The maximum lenght for this field is 10 </div>
</div>
```

Campo do formulário

Mensagens de erro quando a validação falhou

Angular Form Validation

Podemos também adicionar validações customizadas utilizando diretivas em Angular:

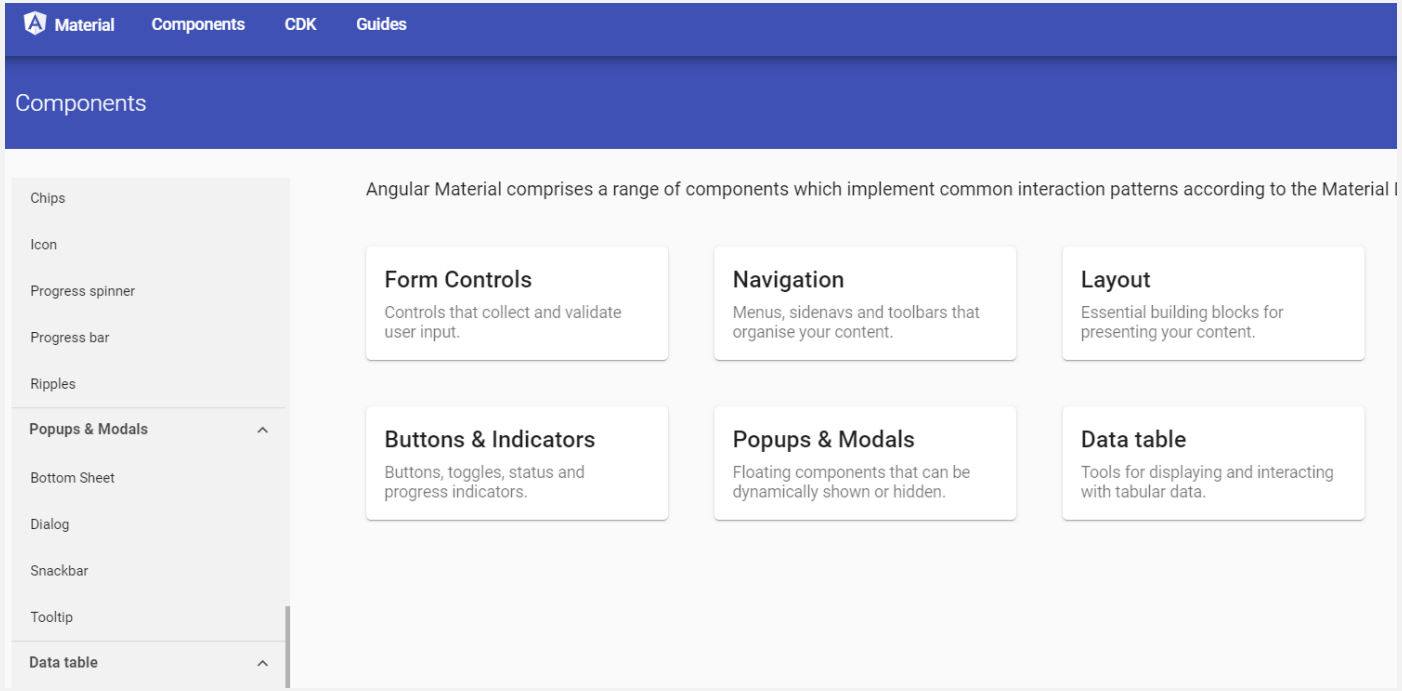
- https://angular.io/guide/form-validation#adding-custom-validators-to-template-driven-forms

Algumas bibliotecas como Angular Material também oferecem validações adicionais sobre alguns dos seus componentes de input

Angular Material

Escolher os componentes a utilizar a partir da lista de componentes em:

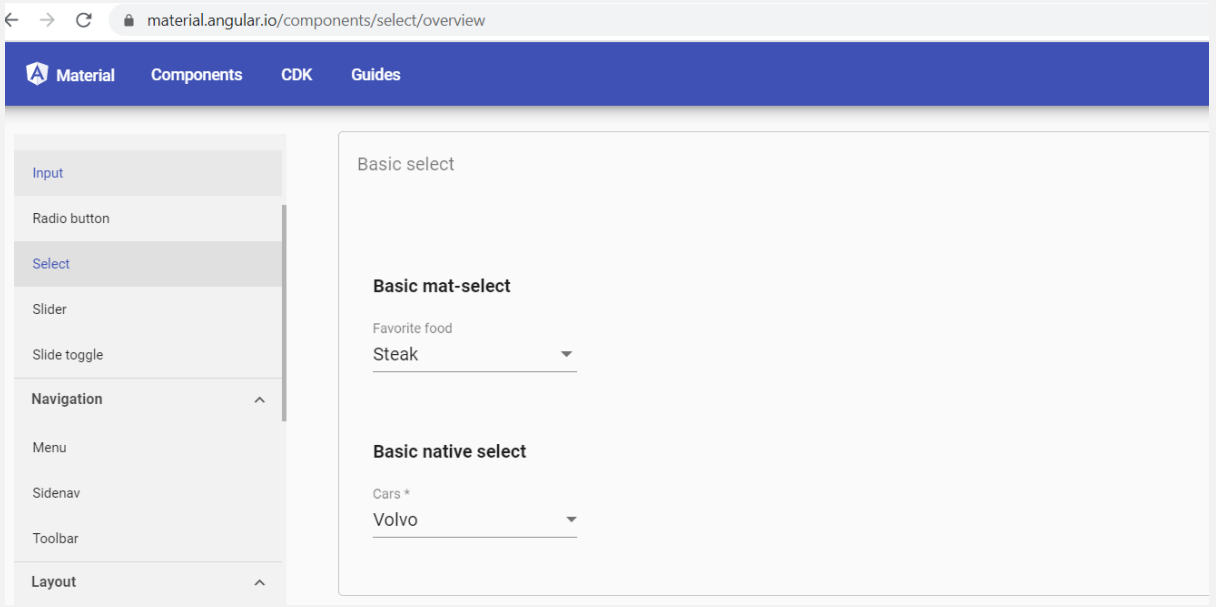
- https://material.angular.io/components/categories



Angular Material

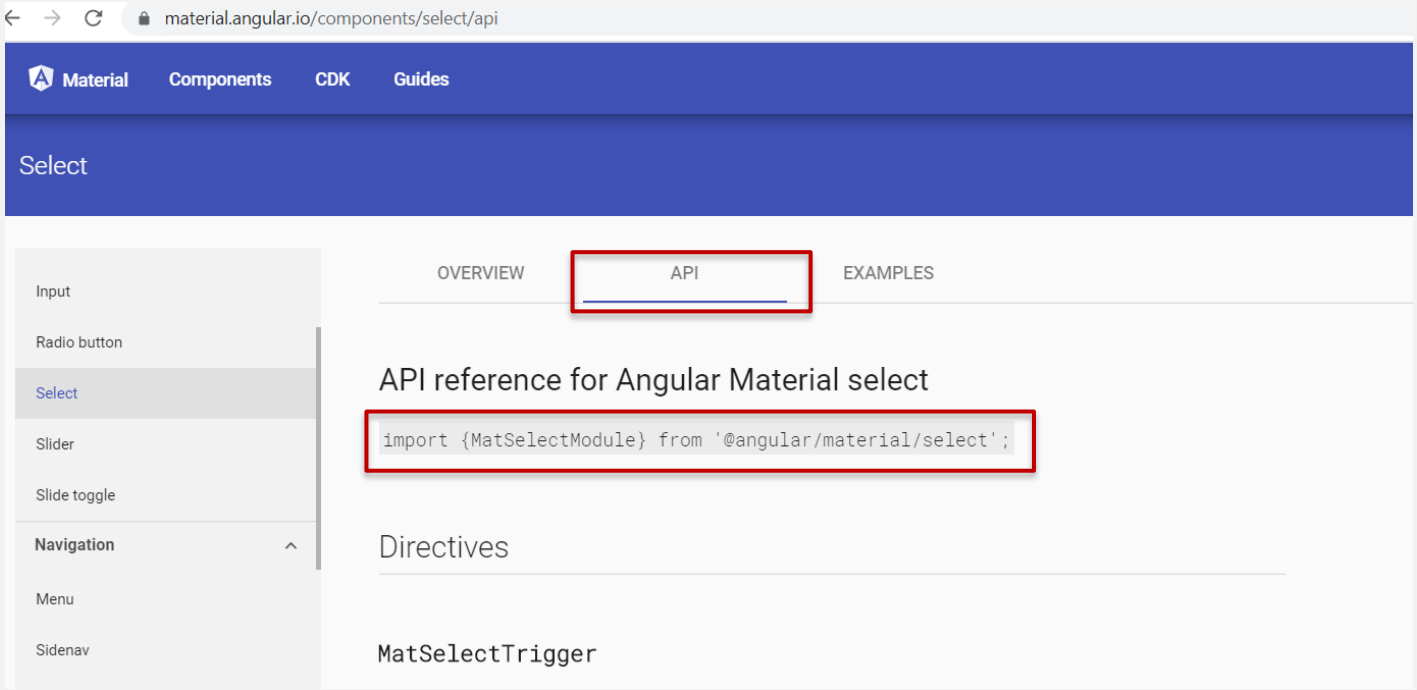
Vamos escolher um componente a título de exemplo:

- Select https://material.angular.io/components/select/overview



Angular Material

Quando um componente é escolhido é necessário adicionar ao ficheiro app.module.ts o import deste módulo



Angular Material

Quando um componente é escolhido é necessário adicionar ao ficheiro app.module.ts o import deste módulo

```
import { BrowserModule } from '@angular/platform-browser';
import { NgModule } from '@angular/core';
import { FormsModule }   from '@angular/forms';

import {MatSelectModule} from '@angular/material/select';

import { AppRoutingModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { NoteAddComponent } from './note-add/note-add.component';
import { NoteListComponent } from './note-list/note-list.component';
import { NotesFullComponent } from './notes-full/notes-full.component';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
import { NotesDialogComponent } from './notes-dialog/notes-dialog.component';

@NgModule({
  declarations: [
    AppComponent,
    NoteAddComponent,
    NoteListComponent,
    NotesFullComponent,
    NotesDialogComponent
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule,
    BrowserAnimationsModule,
    MatSelectModule
  ],
  entryComponents: [
    NotesDialogComponent
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

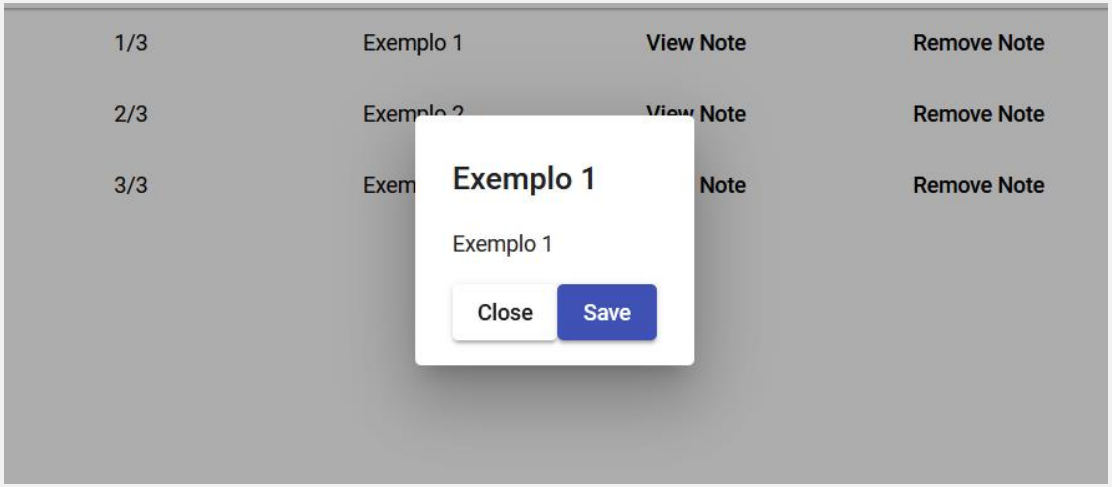
Angular Material

No exemplo da lista de tarefas temos vários usos da biblioteca angular material para:

- Layout da página (mat-card, mat-grid,…)
- Elementos do tipo botão (mat-button, …)
- Elementos do tipo input (mat-input, …)
- Dialogs (MatDialog, ...)

Angular Material

O caso do dialog serve para apresentar informação sobre a página



É um componente dinâmico e comunica com o componente que o invoca quer para receber ou enviar informação

Angular Material

Um dialog é um componente normal em Angular com algumas configurações adicionais


```
<h2 mat-dialog-title>{{note.title}}</h2>

<mat-dialog-content >
   {{note.description}}
</mat-dialog-content>

<mat-dialog-actions>
    <button class="mat-raised-button"(click)="close()">Close</button>
    <button class="mat-raised-button mat-primary"(click)="save()">Save</button>
</mat-dialog-actions>
```

Angular Material

Um dialog é um componente normal em Angular com algumas configurações adicionais


```typescript
@Component({
  selector: 'app-notes-dialog',
  templateUrl: './notes-dialog.component.html',
  styleUrls: ['./notes-dialog.component.css']
})
export class NotesDialogComponent  {

  note :Note;

  constructor(private dialogRef: MatDialogRef<NotesDialogComponent>,
    @Inject(MAT_DIALOG_DATA) public data: DialogData) {
    this.note = data.note;
    console.log(this.note);
  }

  save(): void{
    this.dialogRef.close('closed');
  }

  close(): void{
    this.dialogRef.close();
  }

}

export interface DialogData {
  note: Note;
}
```

Angular Material

O dialog é inicializado através de um objecto MatDialog inicializado com o nosso componente


```typescript
@Component({
  selector: 'app-note-list',
  templateUrl: './note-list.component.html',
  styleUrls: ['./note-list.component.css']
})
export class NoteListComponent {

  notes : Note[];

  constructor(private notesService: NotesServiceService, public dialog: MatDialog) {
  }

  ngOnInit(): void {
    this.notesService.notes.subscribe((notes) =>{ this.notes = notes;
    console.log(JSON.stringify(notes))}, (err) => { console.log(err); });
  }

  openDialog(index:number): void{
    const dialogConfig = new MatDialogConfig();
    dialogConfig.disableClose = true;
    dialogConfig.autoFocus = true;
    dialogConfig.data = {
        note:this.notes[index]
    };
    const dialogRef = this.dialog.open(NotesDialogComponent,dialogConfig);
    dialogRef.afterClosed().subscribe(result => {
      console.log(`Dialog result: ${result}`);
    });
  }

  removeNote(index:number): void{
    this.notesService.deleteNote(index);
  }

}
```

Angular Material

No momento de criação do dialog são passados os inputs e definido o que fazer com o output proveniente do dialog


```typescript
@Component({
  selector: 'app-note-list',
  templateUrl: './note-list.component.html',
  styleUrls: ['./note-list.component.css']
})
export class NoteListComponent {

  notes : Note[];

  constructor(private notesService: NotesServiceService, public dialog: MatDialog) {
  }

  ngOnInit(): void {
    this.notesService.notes.subscribe((notes) =>{ this.notes = notes;
    console.log(JSON.stringify(notes))}, (err) => { console.log(err); });
  }

  openDialog(index:number): void{
    const dialogConfig = new MatDialogConfig();
    dialogConfig.disableClose = true;
    dialogConfig.autoFocus = true;
    dialogConfig.data = {
        note:this.notes[index]
    };
    const dialogRef = this.dialog.open(NotesDialogComponent,dialogConfig);
    dialogRef.afterClosed().subscribe(result => {
      console.log(`Dialog result: ${result}`);
    });
  }

  removeNote(index:number): void{
    this.notesService.deleteNote(index);
  }

}
```

Angular Material

Por fim, é necessário adicionar um campo no ficheiro app.module.ts

Um dialog é considerado um componente dinâmico e por isso necessita desta configuração adicional

```
@NgModule({
  declarations: [
    AppComponent,
    NoteAddComponent,
    NoteListComponent,
    NotesFullComponent,
    NotesDialogComponent,
    NotesDetailsComponent,
    NotesList2Component
  ],
  imports: [
    BrowserModule,
    AppRoutingModule,
    FormsModule,
    BrowserAnimationsModule,
    MatGridListModule,
    MatInputModule,
    MatButtonModule,
    MatCardModule,
    MatDialogModule
  ],
  entryComponents: [
    NotesDialogComponent
  ],
  providers: [],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Deploy Angular

Existem inúmeras formas de efectuar o deploys de uma aplicação Angular:

- AWS
- Azure
- GoogleCloud
- Heroku
- Servidor Próprio
- (outros)

Deploy Angular

No entanto as aplicações em Angular tem uma caraterísticas interessante:

- São aplicações que executam do lado do cliente
- São consideradas estáticas, por que não necessitam de processamento no servidor para executar
  - Podem no entanto comunicar com serviços para obter dados, autenticar utilizadores, …

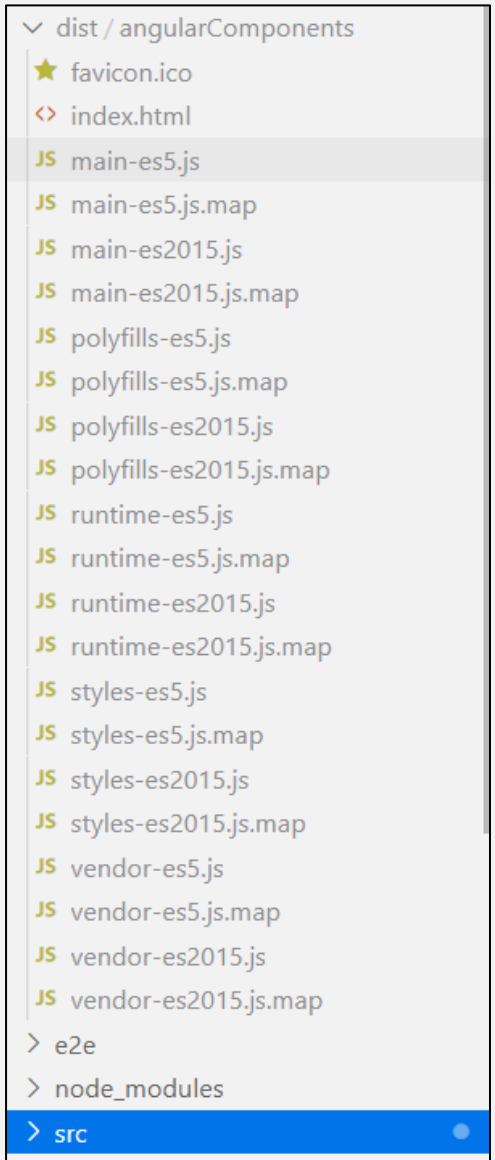
Deploy Angular

A nossa aplicação de Angular no seu formato final pode ser obtida com o comando:

- ng build

A aplicação resultante estará dentro da pasta dist/

A aplicação pode ser iniciada ao abrir o ficheiro index.html no seu browser



Deploy Angular

Devido a estas caraterísticas podemos fazer uso de plataformas que autorizem a publicação de website estáticos:

- Gitlab Pages
- Github Pages
- …

Deploy Angular

Usando a plataforma Gitlab Pages, teremos de:

- Criar um repositório público com o nosso projeto
- Efectuar upload do nosso projeto para o repositório
- Criar um ficheiro .gitlab-ci.yml com o script para efectuar o deploy na nossa aplicação

Deploy Angular

Exemplo do conteúdo do ficheiro

- .gitlab-ci.yml

```
image: node

pages:
  cache:
    paths:
    - node_modules/

  stage: deploy
  script:
  - npm install -g @angular/cli
  - npm install
  - ng build --base-href=/angular_todo/
  - mkdir public
  - mv dist/angularComponents/* public/
  artifacts:
    paths:
    - public
  only:
  - master
  - pages
```

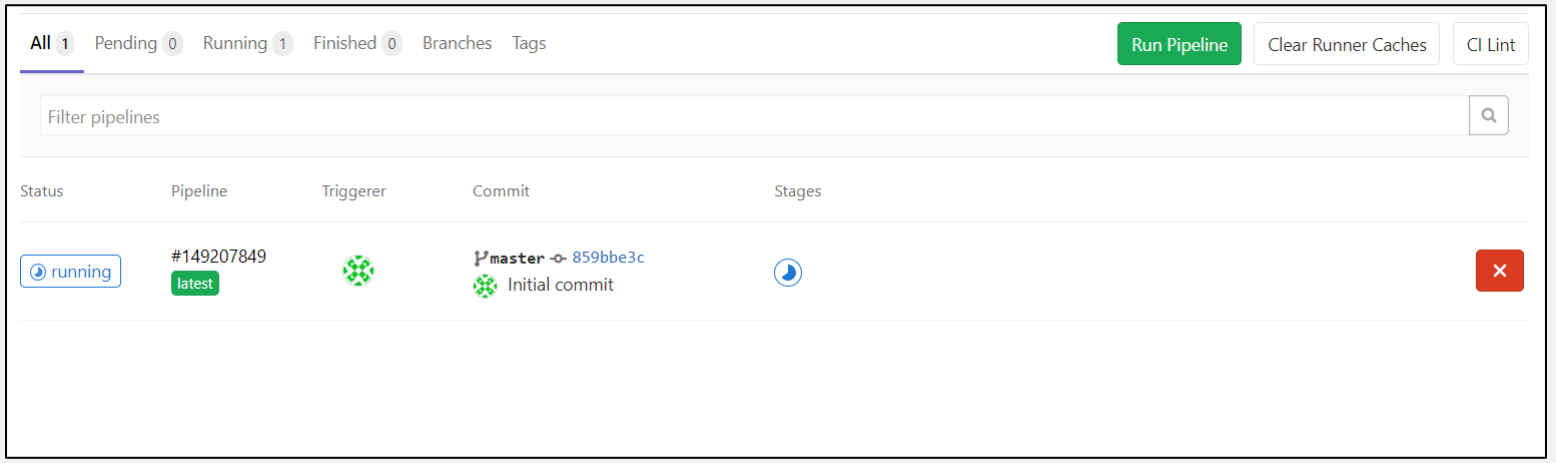
Substituir pelo nome do repositório no gitlab pages

Substituir pelo nome do projecto em Angular

Deploy Angular

Após o commit, a plataforma gitlab irá correr o script e efectuar a build do projeto

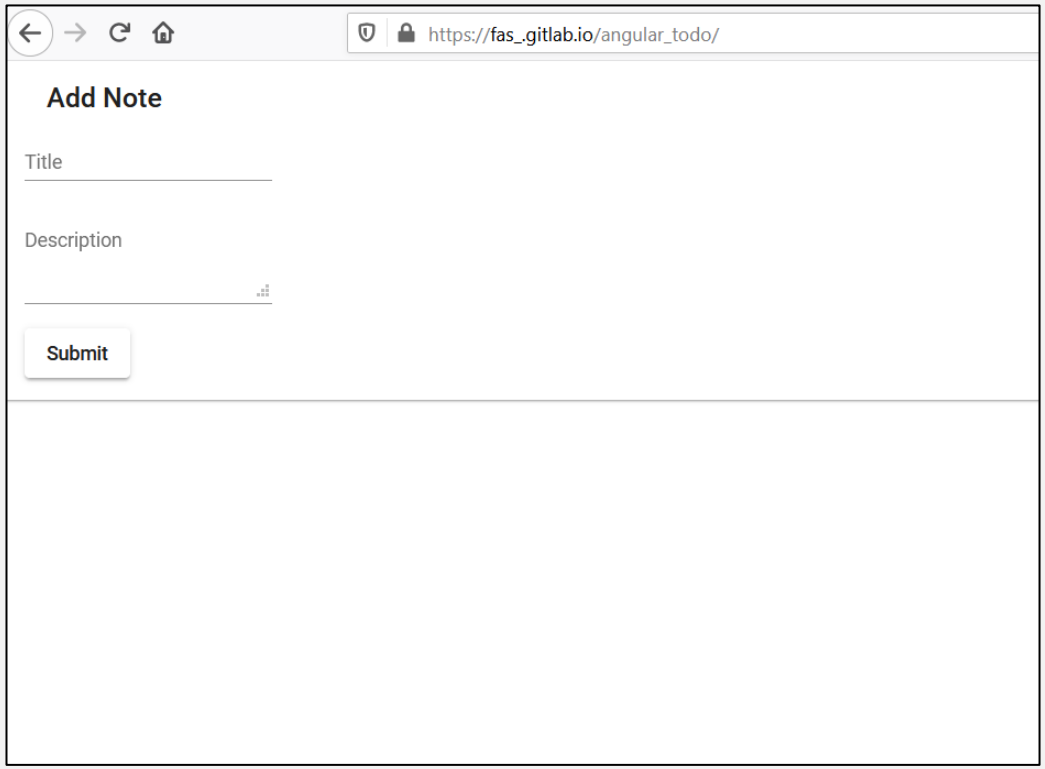
Este processo poderá demorar um bocado!



Deploy Angular

Consultar o endereço:

- https://[gitlabUsername].gitlab.io/[projectName]



Deploy Angular

A partir deste momento temos a aplicação num servidor publico com o domínio associado ao repositório gitlab

A aplicação está também geralmente a usar o protocolo HTTPS como pedido pelas boas práticas de programação web

Deploy Angular

No entanto, e o que fazer acerca de aplicações que necessitem de um servidor, como por exemplo as nossas REST API?

Iremos na próxima aula abordar esse assunto…

Referências

Angular Templates

- https://angular.io/guide/template-syntax

Angular Forms

- https://angular.io/guide/forms-overview

Angular Components

- https://angular.io/guide/component-interaction

Angular Material

- https://material.angular.io/components/categories